

Agentic AI Minimum Guide

Song Kyungmo, Tech University of Korea

2026-04-08

Windows 환경에서 OpenAI API 를 이용한 최소 수준의 Agentic AI 과제 수행 가이드

이 문서는 Windows GUI 사용자 학생이 OpenAI API key 를 발급받아 Python 과 Streamlit 으로 자신의 과업을 수행하는 최소 수준의 agentic AI 프로그램을 실행해보도록 안내하기 위한 실습용 가이드이다.

구성은 다음 5 부분이다.

1. 최소 수준의 app.py 프레임
 2. Python 설치법
 3. 메모장과 VS Code 같은 에디터 기본 사용법
 4. OpenAI API key 발급 및 .env 저장 관리법
 5. Windows Terminal 에서 .venv 가상환경 활성화 후 .py 실행법
-

1. 최소 수준의 app.py 프레임

1-1. 이 프레임의 목적

이 프레임은 다음을 만족하도록 설계하였다.

- 웹 브라우저에서 실행되는 Streamlit 챗봇 화면 제공
- OpenAI API key 를 .env 파일에 보관하고 불러오기
- system, assistant, user 메시지 역할을 명시적으로 구분
- multi-turn 대화 유지
- few-shot prompting 가능
- 학생이 자신의 로컬 문서 txt, md, pdf 와 참고 URL 을 지정 가능
- 각 부분에 교육용 주석 포함

1-2. 권장 프로젝트 폴더 구조

과제를 하는 학생에게는 다음 구조가 가장 단순하다.

```
AI_Agent_Project/  
|  
|-- app.py  
|-- .env
```

```
| -- requirements.txt
| -- README.md
|
| -- data/
|   |-- sample1.txt
|   |-- sample2.md
|   |-- sample3.pdf
|
|-- .venv/
```

1-3. requirements.txt

아래 내용을 requirements.txt 라는 이름으로 저장한다.

```
openai
streamlit
python-dotenv
pypdf
requests
beautifulsoup4
```

1-4. .env

아래처럼 .env 파일을 만든다. 실제 키 값은 학생 본인의 키로 바꾼다.

OPENAI_API_KEY= 여기에 _ 본인의 _API_KEY 를 _ 붙여넣기

1-5. 최소 수준의 app.py 전체 예시

```
# =====
# app.py
# 목적:
# - OpenAI API 를 이용한 최소 수준의 Streamlit 챗봇 예제
# - system / assistant / user 메시지 구조 학습
# - multi-turn 대화 유지
# - few-shot prompting 실험 가능
# - 로컬 참고문서와 참고 URL 내용을 함께 전달 가능
# =====

import os
from io import BytesIO

import requests
import streamlit as st
from bs4 import BeautifulSoup
from dotenv import load_dotenv
from openai import OpenAI
from pypdf import PdfReader
```

```

# 1. 환경변수 로드
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")

if not api_key:
    raise ValueError(
        "OPENAI_API_KEY 를 찾지 못했습니다. .env 파일에 API key 를 저장했는지 확인하세요."
    )

# 2. OpenAI 클라이언트 생성
client = OpenAI(api_key=api_key)

# 3. 참고문서 읽기 함수
def read_uploaded_file(uploaded_file) -> str:
    file_name = uploaded_file.name.lower()

    if file_name.endswith(".txt") or file_name.endswith(".md"):
        return uploaded_file.read().decode("utf-8", errors="ignore")

    elif file_name.endswith(".pdf"):
        pdf_bytes = BytesIO(uploaded_file.read())
        reader = PdfReader(pdf_bytes)
        pages = []
        for page in reader.pages:
            pages.append(page.extract_text() or "")
        return "\n".join(pages)

    else:
        return f"[지원하지 않는 파일 형식: {uploaded_file.name}]"

# 4. 참고 URL 읽기 함수
def read_url_text(url: str) -> str:
    try:
        headers = {"User-Agent": "Mozilla/5.0"}
        response = requests.get(url, headers=headers, timeout=10)
        response.raise_for_status()

        soup = BeautifulSoup(response.text, "html.parser")

        for tag in soup(["script", "style", "noscript"]):
            tag.decompose()

        text = soup.get_text(separator="\n")
        lines = [line.strip() for line in text.splitlines() if line.strip()]
        clean_text = "\n".join(lines)

        return clean_text[:6000]

```

```

except Exception as e:
    return f"[URL 읽기 실패: {url} / 오류: {e}]"

# 5. 메시지 구성 함수
def build_messages(
    system_prompt: str,
    few_shot_user: str,
    few_shot_assistant: str,
    reference_text: str,
    conversation_history: list,
    current_user_input: str,
):
    messages = []

    messages.append({
        "role": "system",
        "content": system_prompt
    })

    if few_shot_user.strip() and few_shot_assistant.strip():
        messages.append({
            "role": "user",
            "content": few_shot_user
        })
        messages.append({
            "role": "assistant",
            "content": few_shot_assistant
        })

    if reference_text.strip():
        messages.append({
            "role": "user",
            "content": (
                " 다음은 참고자료이다. 필요할 때만 활용하고, "
                " 참고자료에 없는 내용은 임의로 단정하지 말라.\n\n"
                f"[참고자료 시작]\n{reference_text}\n[참고자료 끝]"
            )
        })

    messages.extend(conversation_history)

    messages.append({
        "role": "user",
        "content": current_user_input
    })

    return messages

```

```

# 6. Streamlit 화면 설정
st.set_page_config(page_title="My Agentic AI Starter", layout="wide")
st.title("My Agentic AI Starter")
st.caption("OpenAI API + Streamlit + .env + 참고문서/URL + Multi-turn 예제")

# 7. 세션 상태 초기화
if "chat_history" not in st.session_state:
    st.session_state.chat_history = []

# 8. 사이드바 설정
with st.sidebar:
    st.header("설정")

    system_prompt = st.text_area(
        "System Prompt",
        value=(
            "너는 사용자의 과업을 정확하고 구조적으로 돕는 AI assistant 다. "
            "모르면 모른다고 말하고, 근거가 약하면 추정이라고 밝혀라. "
            "답변은 핵심부터 정리하고, 필요한 경우 단계별로 설명하라."
        ),
        height=180,
    )

    st.subheader("Few-shot 예시")
    few_shot_user = st.text_area(
        "예시 user 메시지",
        value="예: 어떤 기업의 사업모델을 3 단 구조로 요약해줘.",
        height=80,
    )
    few_shot_assistant = st.text_area(
        "예시 assistant 메시지",
        value="예: 1) 고객 2) 가치제안 3) 수익구조의 3 단 구조로 요약하겠습니다.",
        height=100,
    )

    model_name = st.text_input("모델 이름", value="gpt-5")

    st.markdown("---")
    st.subheader("참고문서 업로드")
    uploaded_files = st.file_uploader(
        "txt / md / pdf 파일 업로드",
        type=["txt", "md", "pdf"],
        accept_multiple_files=True,
    )

    st.subheader("참고 URL 입력")
    url_text = st.text_area(
        "URL 을 한 줄에 하나씩 입력",

```

```

        value="",
        height=120,
        placeholder="https://example.com/article1\nhttps://example.com/article2",
    )

    clear_button = st.button(" 대화 기록 초기화")
    if clear_button:
        st.session_state.chat_history = []
        st.success(" 대화 기록을 지웠습니다.")

# 9. 참고자료 결합
reference_parts = []

if uploaded_files:
    for file in uploaded_files:
        file_text = read_uploaded_file(file)
        reference_parts.append(f"\n[로컬 파일: {file.name}]\n{file_text[:6000]}")

if url_text.strip():
    urls = [u.strip() for u in url_text.splitlines() if u.strip()]
    for url in urls:
        url_content = read_url_text(url)
        reference_parts.append(f"\n[참고 URL: {url}]\n{url_content}")

reference_text = "\n\n".join(reference_parts)

# 10. 이전 대화 출력
for msg in st.session_state.chat_history:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

# 11. 현재 사용자 입력
user_input = st.chat_input(" 질문이나 과업 지시를 입력하세요.")

if user_input:
    with st.chat_message("user"):
        st.markdown(user_input)

    messages = build_messages(
        system_prompt=system_prompt,
        few_shot_user=few_shot_user,
        few_shot_assistant=few_shot_assistant,
        reference_text=reference_text,
        conversation_history=st.session_state.chat_history,
        current_user_input=user_input,
    )

    try:

```

```

response = client.chat.completions.create(
    model=model_name,
    messages=messages,
    temperature=0.3,
)
assistant_reply = response.choices[0].message.content

except Exception as e:
    assistant_reply = f" 오류가 발생했습니다: {e}"

with st.chat_message("assistant"):
    st.markdown(assistant_reply)

st.session_state.chat_history.append({
    "role": "user",
    "content": user_input
})
st.session_state.chat_history.append({
    "role": "assistant",
    "content": assistant_reply
})

```

1-6. 학생이 직접 수정해야 할 핵심 위치

A. 챗봇의 성격과 원칙 설정 system_prompt 기본값 부분을 바꾼다.

예: - 너는 대학생의 발표자료 구성을 도와주는 assistant 다. - 너는 지역 문화유산 답사 계획을 짜주는 assistant 다. - 너는 중소기업 마케팅 아이디어를 구조화해주는 assistant 다.

B. few-shot 예시 바꾸기 아래 두 칸은 사용자가 이렇게 물으면 assistant 는 이런 형식으로 답 하라는 예시이다.

```

few_shot_user = st.text_area(...)
few_shot_assistant = st.text_area(...)

```

C. 모델 이름 바꾸기

```
model_name = st.text_input(" 모델 이름", value="gpt-5")
```

강의 시점의 사용 가능 모델명으로 바뀌서 쓴다.

D. 실제 과업 입력 웹 화면 하단 입력창에 학생이 자기 과업을 입력한다.

예: - 내가 업로드한 PDF 를 바탕으로 5 분 발표용 요약을 만들어줘. - 첨부 자료를 보고 핵심 개념 7 개를 뽑아줘. - 참고 URL 내용을 비교해서 장단점을 표로 정리해줘.

2. Python 설치법 안내

2-1. Windows 에서 별도 Linux 나 WSL 없이 가능한가

가능하다. 학생 과제 수준의 OpenAI API 와 Streamlit 앱은 Windows 용 Python 만으로 충분히 수행 가능하다.

2-2. 설치 절차

1. Python 공식 사이트에 들어간다.
2. Windows 용 설치 파일을 내려받는다.
3. 설치 프로그램 실행 시 Add python.exe to PATH 를 체크한다.
4. 설치 완료 후 Windows Terminal 또는 명령 프롬프트를 연다.
5. 아래 명령으로 설치를 확인한다.

```
python --version
```

또는

```
py --version
```

3. 메모장과 VS Code 같은 에디터 기본 사용법 안내

3-1. 최소 수준: 메모장

정말 최소 수준이라면 학생은 다음만 할 수 있어도 된다.

- 새 텍스트 파일 열기
- ChatGPT 가 생성해준 코드 붙여넣기
- 파일명을 app.py 로 저장하기
- 같은 방식으로 .env, requirements.txt 저장하기

단, 저장 시 아래를 지켜야 한다.

- 파일 형식: 모든 파일
- 파일명: app.py
- 인코딩: UTF-8 권장

3-2. 권장 수준: VS Code

학생에게는 가능하면 VS Code 를 권장하는 편이 낫다.

이유: - .py, .env, .txt 파일을 구분해서 보기 쉽다. - 들여쓰기와 괄호를 보기 쉽다. - 저장 실수가 적다. - 오류 위치를 찾기 쉽다.

3-3. VS Code 에서 필요한 최소 동작

1. 폴더 열기
2. 새 파일 만들기
3. 코드 붙여넣기 후 저장
4. 파일명 확인

5. 터미널 열기

4. OpenAI API key 발급 및 .env 저장 관리법

4-1. 핵심 원칙

API key 는 비밀번호와 같은 비밀정보다.

절대 하면 안 되는 것: - app.py 안에 key 를 직접 입력 - 과제 제출 파일 안에 key 포함 - 메신저나 이메일로 key 공유 - 공개 저장소에 업로드

해야 하는 것: - .env 파일에만 저장 - 타인에게 보여주지 않기 - 유출 의심 시 즉시 폐기하고 새 key 발급

4-2. .env 예시

```
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

4-3. .env 를 Python 에서 읽는 구조

```
from dotenv import load_dotenv
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
```

5. Windows Terminal 에서 .venv 활성화 후 .py 실행하는 법

5-1. 왜 .venv 를 쓰는가

이유는 다음과 같다.

1. 프로젝트별 라이브러리 충돌 방지
2. PC 전체 Python 환경을 복잡하게 만들지 않음
3. 오류가 나면 .venv 를 지우고 다시 만들기 쉬움
4. 과제별 재현성이 좋아짐

5-2. 프로젝트 폴더로 이동

```
cd D:\AI_Agent_Project
```

또는

```
cd %HOME%\Desktop\AI_Agent_Project
```

5-3. 가상환경 만들기

```
py -m venv .venv
```

5-4. 가상환경 활성화

```
.venv\Scripts\activate
```

정상 활성화되면 보통 프롬프트 앞에 (.venv) 가 보인다.

5-5. pip 업데이트

```
python -m pip install --upgrade pip
```

5-6. 필요한 라이브러리 설치

```
pip install -r requirements.txt
```

5-7. Streamlit 앱 실행

```
streamlit run app.py
```

5-8. 종료와 재실행

앱을 멈출 때:

```
Ctrl + C
```

가상환경을 빠져나올 때:

```
deactivate
```

다음번에 다시 실행할 때:

```
cd D:\AI_Agent_Project  
.venv\Scripts\activate  
streamlit run app.py
```

5-9. 자주 만나는 오류

오류 1. python 또는 py 를 찾지 못함 원인: - Python 설치가 안 되었거나 PATH 설정이 안 되었음

조치: - Python 재설치 - 설치 시 PATH 추가 체크

오류 2. OPENAI_API_KEY 를 찾지 못했습니다 원인: - .env 파일 위치가 틀림 - 파일명이 .env.txt 로 저장됨 - 변수명이 잘못됨

조치: - 프로젝트 폴더 안에 정확히 .env 가 있는지 확인 - 내용이 OPENAI_API_KEY=... 형식 인지 확인

오류 3. ModuleNotFoundError 원인: - 필요한 라이브러리가 설치되지 않음 - 가상환경을 활성화하지 않은 채 실행함

조치:

```
.venv\Scripts\activate  
pip install -r requirements.txt
```

오류 4. streamlit 명령을 찾지 못함 원인: - 가상환경 비활성 상태 - Streamlit 미설치

조치:

```
.venv\Scripts\activate  
pip install streamlit
```

6. 학생 과제 수행 절차 요약

1. Python 설치
 2. 프로젝트 폴더 생성
 3. app.py, .env, requirements.txt 저장
 4. Windows Terminal 에서 프로젝트 폴더로 이동
 5. `py -m venv .venv`
 6. `.venv\Scripts\activate`
 7. `pip install -r requirements.txt`
 8. `streamlit run app.py`
 9. 웹 화면에서 자기 과업 입력
 10. 필요하면 ChatGPT 에게 app.py 수정 요청
 11. 수정된 코드를 다시 저장하고 재실행
-

7. 교수자용 수업 운영 팁

학생들에게는 처음부터 완전한 agent 를 요구하기보다, 다음 3 단계로 과제를 설계하는 것이 현실적이다.

1 단계

기본 프레임 실행 성공 - .env 연결 - Streamlit 화면 실행 - 한 번의 질문과 응답 성공

2 단계

과업 맞춤화 - system prompt 수정 - few-shot 예시 수정 - 참고 문서 업로드 실험 - 참고 URL 결합 실험

3 단계

agentic 확장 - 함수 추가 - 계산, 분류, 요약 자동화 - 파일 처리 자동화 - 필요 시 function calling 구조 추가

8. 마무리

핵심은 다음 셋이다.

- 코드 전체를 처음부터 스스로 작성하려 하지 말 것
- 기본 프레임을 먼저 실행해볼 것
- 이후 ChatGPT 를 이용해 자신의 과업에 맞게 반복 수정할 것

이 정도 수준만 확보해도 학생은 프롬프트 사용자를 넘어, 자기 과업을 수행하는 간단한 AI 도구를 직접 조립하는 사용자로 넘어갈 수 있다.